

Surge

Operations Runbook

Apple M2 Mac mini · hcucoses-Mac-mini · code at /opt/surge

Written for an instructor, TA, or the next student, assuming no prior contact with the project. If you read only one section, read **Is it working?** and **When something is broken**.

What this thing does

Twice a day (09:00 and 15:00, Mon–Fri) it runs an automated network scan of the lab subnet, correlates what it finds against CVE data, scores the results with a machine-learning model, and writes a report. A web dashboard shows the results.

The scanning is done by `nmap`. The analysis and report writing are done by a large language model reached over the internet (OpenRouter), paid from a prepaid balance. There is a local fallback model for when that balance runs out.

The moving parts

SERVICE	WHAT IT IS	RUNS AS	STARTS AT BOOT
<code>homebrew.mxcl.postgresql@16</code>	the database	<code>hcu.cose</code>	yes
<code>com.surge.api</code>	backend + scan engine, port 8000	<code>root</code>	yes
<code>com.surge.web</code>	dashboard, port 3000	<code>hcu.cose</code>	yes
<code>com.surge.schedule</code>	fires the 09:00 / 15:00 scans	<code>hcu.cose</code>	on schedule only
<code>homebrew.mxcl.ollama</code>	local fallback model	<code>hcu.cose</code>	yes

All are launchd jobs defined in `/Library/LaunchDaemons/`.

The API runs as root on purpose. nmap needs raw network sockets for SYN scans and OS fingerprinting. Without root it silently finds nothing — which looks exactly like an empty network rather than an error. Do not "fix" this by running it as a normal user.

Is it working?

```
sudo launchctl list | grep surge
curl http://localhost:8000/health      # {"status":"ok"}
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000    # 200
psql -U surge -d surge -c "\dt"      # 6 tables
```

Dashboard: **http://localhost:3000** in a browser on the mini itself.

Recent scans:

```
psql -U surge -d surge -c \
  "select left(scan_id,8) id, status, devices_count, vulns_count, round(duration_s)
  secs, created_at \
  from scans order by created_at desc limit 5;"
```

A healthy scan ends **completed** with **devices_count** greater than zero.

Where the logs are

<code>/var/log/surge-api.err.log</code>	backend + every scan. Start here.
<code>/var/log/surge-schedule.log</code>	why a scheduled run did or didn't start
<code>/var/log/surge-web.err.log</code>	dashboard
<code>/opt/homebrew/var/log/postgresql@16.log</code>	database
<code>/opt/surge/surge-ai/report/<timestamp>_<scanid>/</code>	that scan's raw output and reports

When something is broken

Nothing responds at all

```
sudo launchctl list | grep surge
sudo launchctl print system/com.surge.api | grep -Ei "state|last exit"
```

state = running is healthy. **spawn scheduled** means it is crash-looping — read `/var/log/surge-api.err.log`.

The dashboard loads but has no data

Almost always the database. Check it is up:

```
brew services list | grep postgres
psql -U surge -d surge -c "select count(*) from scans;"
```

If Postgres is down, the backend crash-loops until it returns. It self-heals once the database is back — no need to restart anything by hand.

A scan says **completed** but found 0 devices

This is the signature of nmap running **without root**. Confirm the API daemon is running as root:

```
ps -o user,command -p $(sudo launchctl print system/com.surge.api | awk
'/pid =/{print $3}')
```

It must say **root**. Also confirm you are scanning a subnet the mini is actually attached to — the mini has two network interfaces:

```
ifconfig en0 | grep "inet "      # campus network
ifconfig en1 | grep "inet "      # lab network — this is the one being scanned
```

A scan is stuck at **running** forever

The backend marks orphaned scans as **failed** on startup, so:

```
sudo launchctl kickstart -k system/com.surge.api
```

The scheduled run didn't happen

```
tail -40 /var/log/surge-schedule.log
```

Expected reasons it skips, both normal and logged plainly:

- another scan was still running (deliberate — the two slots are 6 h apart but a scan can run longer)
- the API was unreachable

A service ignores your edits to its .plist

launchctl print shows what launchd is **actually running**. The .plist file shows what you **meant**. When they disagree, the file is a lie — launchd caches a job's configuration when it is first bootstrapped and never re-reads it, and **kickstart** only restarts the process with that cached copy.

Compare them:

```
sudo launchctl print system/<job> | grep -E "username|environment" -A6
/usr/libexec/PlistBuddy -c "Print" /Library/LaunchDaemons/<job>.plist
```

Reload properly — the `sleep` matters, unloading is asynchronous:

```
sudo launchctl bootout system/<job>
sleep 2
sudo launchctl bootstrap system /Library/LaunchDaemons/<job>.plist
```

This is why the 09:00 job silently failed for a day: the plist said `UserName = hcu.cose`, but `launchd` was still holding an earlier value naming a user that did not exist, so every firing died instantly with `EX_CONFIG`. Use `kickstart` after a `code` change; use `bootout`/`bootstrap` after a `configuration` change.

A service won't start, exit code 78 (`EX_CONFIG`), and there is no log

This has a single cause, and the missing log is the clue. Any `launchd` job with a `UserName` set runs as that user, but `launchd` opens its `StandardOutPath` / `StandardErrorPath` as that user, **before running the program**. If the log file is owned by root, the open fails, `launchd` aborts the job, and nothing gets written — because writing the log is what failed.

Find who owns the log the job wants:

```
/usr/libexec/PlistBuddy -c "Print :StandardOutPath" /Library/LaunchDaemons/<job>.plist
ls -la <that path>
```

Fix by handing the file to the user the job runs as:

```
sudo chown hcu.cose:admin <that path>
sudo launchctl kickstart -k system/<job>
```

Bitten `com.surge.web` and Ollama. It appears whenever something is installed or started with `sudo` and later switched to run as a normal user.

The backend keeps restarting, exit code 3

Exit 3 means the server never finished starting. Two common causes:

- **Port 8000 already in use.** Someone left a manual `uvicorn` running: `sudo lsof -i :8000`, then kill the stray process.
- **Database unreachable.** See above.

Reports are blank in the dashboard

Reports for a scan are stored in the database. The *final* report is written automatically when a scan completes; the executive, technical and public variants are generated the first time

somebody asks for them, which takes roughly 20 seconds and costs a small amount of API credit. That delay on first open is expected. Second and later opens are instant.

Money

Scans cost roughly **\$0.015 each** in API credits. Check the balance:

```
cd /opt/surge/surge-ai && ./venv/bin/python check_credits.py
```

If the balance falls below **\$2**, the scheduler automatically switches to the local model instead of failing. Scans keep running but the analysis quality drops noticeably. To top up, add credit to the OpenRouter account tied to the key in `/opt/surge/surge-ai/.env`.

Running a scan by hand

```
cd /opt/surge/surge-ai
sudo -u hcucose ./venv/bin/python /opt/surge/deploy/macmini/scheduled_scan.py
```

Or with an explicit target and type:

```
SURGE_TARGET=10.10.163.0/24 SURGE_SCAN_TYPE=quick \
./venv/bin/python /opt/surge/deploy/macmini/scheduled_scan.py
```

Scan types are `quick`, `normal`, `deep`. A "quick" scan is not necessarily quick — the scanning agent escalates on its own when it finds services, and a run of 30–45 minutes is normal. `deep` can take several hours.

Restarting things

```
sudo launchctl kickstart -k system/com.surge.api          # after a code change
sudo launchctl kickstart -k system/com.surge.web
```

If you edit a `.plist` file, **kickstart** is not enough — `launchd` caches the job configuration and will silently keep using the old one:

```
sudo launchctl bootout system/com.surge.api
sleep 2
sudo launchctl bootstrap system /Library/LaunchDaemons/com.surge.api.plist
```

The **sleep** matters. Unloading is asynchronous, and bootstrapping too quickly fails with an unhelpful **Input/output error**. That same error also appears when a service is *already* loaded, so it rarely means what it seems to.

After a power cut

The mini is configured to power itself back on and bring everything up without anyone logging in. Verify:

```
pmset -g | grep autorestart      # must be 1
fdesetup status                  # must be Off
```

FileVault must stay off. With it on, the mini reboots to a disk-unlock prompt and no service starts until someone types a password at the physical console — the machine looks powered on while Surge is entirely dead.

The database and backend start independently, so on a cold boot the backend may restart a few times before Postgres is ready. That is expected and self-correcting.

Updating the code

```
sudo git -C /opt/surge pull
sudo launchctl kickstart -k system/com.surge.api
```

The repository is owned by root, so **git** needs **sudo**. If it complains about "dubious ownership":

```
sudo git config --global --add safe.directory /opt/surge
```

Things that are known and expected

Not bugs. Do not spend time on them.

- **InconsistentVersionWarning from scikit-learn** on every start. The model encoders were saved with an older version. Verified working; there is a startup check that would fail loudly if they ever genuinely broke.
- **A "quick" scan taking 30–45 minutes.** The agent escalates when it finds services.
- **devices_count staying 0 while a scan runs.** Devices are written to the database when the scan finishes, not as it goes.
- **Bootstrap failed: 5: Input/output error** usually means the service is already loaded. Check with **launchctl print** before assuming it failed.

Not yet proven

Honest gaps, so nobody is surprised:

- **The ML scoring path has never run on real vulnerability data.** Every test scan so far found hosts but no CVEs, so the scoring model has not actually produced a score in production. It previously crashed the process here; that crash is fixed and verified in isolation, but not yet under a live scan with findings. **The first scan that reports a non-zero `vulns_count` is the real test.** If the backend dies during scoring, check `/var/log/surge-api.err.log` for a crash in `libomp`.
- **Recovery from an actual power cut has not been tested**, only configured.